

ORIGAMI AS A SPATIAL AND GEOMETRIC REASONING BENCHMARK

Anonymous authors

Paper under double-blind review

ABSTRACT

Multimodal language models perform well on visual recognition and on tool use, but these abilities do not transfer directly to spatial and geometric reasoning. Geometric structure does not reduce to text or to a single image: exact angles and incidences, which surface lies in front of which, and the order in which operations were applied are not recoverable from a semantic description of a scene. We introduce **CP2Seq**, a benchmark for evaluating geometric reasoning and sequential planning in multimodal language models through origami. The benchmark contains 600 procedurally generated samples, each requiring a model to produce a valid sequence of folds from a crease pattern and a target folded state. Samples are produced by a deterministic folding engine. Ground truth is recorded as each fold happens rather than annotated afterwards. Every sample rebuilds byte-identically from its seed, so the corpus is released as a generator and a manifest rather than as a data archive. We release the generation and evaluation framework alongside it, so the corpus can be regenerated, extended, or re-stratified at a different difficulty. During evaluation, the engine executes proposed folds or rejects invalid actions with explicit feedback, while the model remains responsible for selecting actions and searching for a solution. We evaluate multimodal language models under progressively greater tool assistance, and against a deterministic breadth-first search over the same action set. Candidate sequences are assessed by executing them and comparing the resulting folded state with the target, allowing for planar translations, rotations, and reflections. Under the low reasoning effort setting, none of the evaluated models solves a sample outside the easy subset. On that subset, the best model achieves a success rate of 20.4%, compared with 54.7% for the search baseline. Its dominant failure mode is not an exhausted query budget but the repeated proposal of states it has already visited, which terminates 48% of its attempts. These results identify a substantial gap between multimodal language models and explicit search on this origami planning task.

1 INTRODUCTION

Geometric planning requires a model to connect a description of a target with the operations needed to construct it. In origami, this involves choosing a fold line, identifying which parts of the sheet should move, and tracking how the resulting reflections change their positions and layer order. A locally valid fold may lead away from the target, so completing a sequence also requires selecting among alternatives and revising earlier decisions. We study whether multimodal language models can use visual and geometric observations to make these decisions over successive folds.

Existing origami benchmarks evaluate several components of this ability. GamiBench (Spencer et al., 2025) uses visual question answering to assess folded configurations, viewpoint consistency, and impossible patterns. OrigamiSpace (Xu et al., 2025) includes pattern prediction, multi-step spatial reasoning, spatial relationships, and crease-pattern code generation with an interactive environment. Our focus is the recovery of an executable folding sequence from its endpoints: the model must construct and revise a sequence whose execution reproduces a supplied crease pattern and target folded state. This task evaluates the consequences of the model’s chosen actions throughout an episode, as well as the correctness of its final construction.

We introduce **CP2Seq**, a benchmark of 600 procedurally generated origami samples and a framework for generating, executing, and evaluating folding sequences. Each task supplies a crease pattern and a target folded state; the reference sequence and intermediate reference states are withheld. Starting from an unfolded square, the model must produce a valid sequence of simple folds under the specified action model. In the visual setting, it receives images of both inputs and views of its current folded state. The crease pattern constrains which creases must be produced, while the target specifies the arrangement of the folded sheet, including layer order. Together, these inputs guide the model’s choice of where to fold and which layers to move.

The task combines *geometric reasoning*, *spatial state tracking*, and *sequential planning*. A fold line is specified in the current folded coordinates, whereas the target crease pattern describes the original sheet. Connecting these representations requires tracking reflections and the correspondence between paper regions. Selecting a fold also requires reasoning about overlapping layers and their connectivity. Subsequent decisions depend on the state produced by earlier actions, so increasing sequence depth extends the planning horizon. CP2Seq evaluates these abilities jointly through successful construction; a failed attempt alone does not identify which ability failed. The represented states are flat, with explicit layer ordering, and the task excludes material mechanics and continuous collision checking.

Procedural generation lets us vary the task distribution while retaining a recorded solution. The generator applies legal folds, records the sequence and intermediate states, and derives the resulting crease pattern. Its configuration controls reference sequence depth and the permitted action model, including all-layers and some-layers folds. Seeded sampling produces different instances, and recorded coupling measures how many creases each fold creates. These controls support evaluation across sequence lengths and geometric structures without manually annotating solutions. Reference depth is a generation parameter, not a guarantee of minimum solution length; likewise, variation across seeds does not by itself establish greater difficulty. The released generator and manifest allow the corpus to be reproduced and extended (Section 4).

The evaluation framework supplies an interactive loop for proposing, checking, and revising folds. Tools let the model add a fold, remove an earlier fold and replay the remaining sequence, inspect geometry and rendered views, and return to an earlier state. Each proposed edit is checked by the folding engine. For example, a fold that moves paper attached to stationary paper away from the hinge is rejected because it would tear the sheet under the supported model. This feedback prevents an invalid proposal from silently becoming the next state. In the filtered condition, a tool also enumerates legal folds compatible with the target crease pattern. This removes part of the action-selection burden, but leaves the model to choose which candidate to explore and when to backtrack. The framework therefore makes tool assistance an explicit experimental condition; success with filtering and success without it test different demands on the model.

Evaluation checks both individual actions and the completed construction. Candidate sequences are replayed and assessed for crease-pattern agreement and agreement with the target folded state. A sequence need not match the recorded actions to be correct. Folded-state comparison accounts for a global translation, rotation, or turnover of the completed object, with the corresponding changes to layer order and face orientation, while preserving the identity of each paper region (Section 6). We also provide a deterministic breadth-first search baseline using the same candidate-fold enumerator and terminal-state comparator. This baseline measures how much can be achieved by systematic search with the available filtering, providing context for model performance beyond comparisons between language models.

Our contributions are:

1. **A reproducible dataset and generation framework.** CP2Seq contains 600 samples with crease patterns, target states, recorded fold sequences, and intermediate states. Configurable generation and seeded reconstruction support experiments across reference depths and action models, with coupling recorded to characterize geometric structure.
2. **An interactive evaluation and verification pipeline.** The framework connects visual and geometric inputs to tools for constructing and revising sequences, step-level legality checks, and final replay-based scoring. It includes corpus checks and a folded-state comparator tested against deliberately corrupted layer arrangements. It defines four assistance conditions; the current experiments cover the filtered and no-tools conditions.

3. **A search baseline and an empirical analysis of model behavior.** We provide breadth-first search over the same candidate actions, report performance by difficulty stratum, and analyze episode termination. The current evaluation concentrates on `gpt-5.6-luna`; other model runs have insufficient coverage for a cross-model ranking.

In the current aggregate results, breadth-first search solves 54.7% of easy-stratum attempts, compared with 20.4% for `gpt-5.6-luna` with filtering and 6.2% without tools. These rates summarize different sets of attempts rather than a paired comparison on an identical sample set. No evaluated language-model attempt succeeds on the medium or hard strata. All recorded reasoning-effort settings are low, so these results do not establish a general limit on the evaluated models. State cycling terminates 48.1% of Luna’s attempts, motivating further experiments on state tracking and backtracking. Section 9 reports the coverage, budgets, and limitations of these measurements. The findings motivate evaluating geometric planning through both the constructions models produce and the search behavior that leads to them.

2 RELATED WORK

2.1 ORIGAMI BENCHMARKS FOR MULTIMODAL MODELS

OrigamiSpace (Xu et al., 2025) contributes 350 instances, each carrying a crease pattern diagram, a compiled flat pattern, the complete folding process and a final folded image, and defines four tasks over them: pattern prediction, multi-step spatial reasoning, spatial relationship prediction, and end-to-end crease-pattern code generation. It also provides an interactive environment and describes reinforcement learning as a possibility explored rather than a result demonstrated. Its design lesson is that one richly instrumented sample can support several tasks, which is how 350 instances sustain a benchmark paper.

GamiBench (Spencer et al., 2025) contributes 186 regular and 186 *impossible* crease patterns paired with folded shapes from six viewpoints, across three visual question-answering tasks, and introduces two diagnostic metrics: viewpoint consistency and impossible fold selection rate. Its impossible patterns are the closest the current literature comes to forcing a model to judge feasibility rather than recognise a shape, and its headline finding is that leading models struggle even at single-step spatial understanding. It is the benchmark this work is most directly in conversation with, and the one whose negative examples we can produce without limit, because generating an illegal fold in our environment is the same operation as generating a legal one.

OrigamiBench (Agarwal et al., 2026), **FoldingAgent** (Moriya et al., 2026) and **COrigami** (Zahavy et al., 2026) complete the picture; their metrics appear in the comparison of Section 2.4.

What all of these share is the gap. In each, the model’s proposal is answered by a question rather than by an execution: it selects among rendered alternatives, or produces an artifact that is compared to a target. None of them steps a paper-exact engine and asks whether *this* move, on *this* stack, is something paper could do.

2.2 METHODS THAT PAIR A PROPOSER WITH A SIMULATOR

Learn2Fold (Huang et al., 2026) formulates origami as conditional program induction over a crease-pattern graph. A language model generates candidate folding programs from text, and a learned graph-structured world model acts as a differentiable surrogate simulator that predicts physical feasibility and failure modes before execution, inside a lookahead planning loop. Its stated key insight is to decouple semantic proposal from physical verification.

We adopt that decomposition and differ on one thing: the verifier. Theirs is learned, differentiable and approximate, which is what makes it usable for planning and also what makes its mistakes invisible. Ours is exact, because it is the generator run forwards. Their scoring is step-level precision, recall and F1 against an expert sequence, together with edge-level IoU; Section 6.1 shows why that family of measures is unsafe in this task, since a model that finds a *shorter* correct sequence is marked wrong by it. That is a finding about scoring in this setting rather than a criticism of their results.

The general precedent for a ((((((proposer paired with a verifier)))))) - —what’s this??? what’s pur proposer is older than any of this work: a network that proposes and a search that checks is the structure behind AlphaGo (Silver et al., 2016), and the reason the combination is stronger than either half.

2.3 COMPUTATIONAL ORIGAMI: THE THEORY UNDER THE ACTION SPACE

The benchmark’s central claim, that geometry does not determine a folded state, is Bern and Hayes’s: deciding the overlap order of a flat folding is NP hard even given a valid mountain-valley assignment (Bern & Hayes, 1996). The action space is Arkin et al.’s: simple folds, in the one-layer, some-layers and all-layers variants, with finite or infinite fold lines (Arkin et al., 2004). Their complexity has been mapped since, including simple foldability’s hardness (Akitaya et al., 2017) and the infinite all-layers case (Akitaya et al., 2019). The solver whose constraint vocabulary we borrow for terminal states is Flat-Folder (Akitaya et al., 2024).

Two results shape what the task actually is. Demaine et al. (2004) showed that any well-behaved folded state is reachable by *some* continuous motion; the question is therefore never whether a path exists, but whether one exists through a finite number of the discrete operations a hand or a machine can perform. And counting foldings has no closed form even for a one-dimensional strip, a problem open since Lucas attributed it to Lemoine in 1891 (Lucas, 1891; OEIS Foundation Inc., 2026); this is the answer to “why not enumerate the states”, and it is a fact about the problem rather than about any implementation.

Finally, the problem itself is not new: generating folding sequences from crease patterns was named and attacked symbolically by Akitaya et al. (2013), by the same group whose tools this field now depends on. What is new is neither the problem nor the theory, but the observation that this problem supplies an exact verifier and can therefore serve as an evaluation substrate for models that reason with tools.

2.4 THE STRUCTURAL GAP

The two closest benchmarks are worth setting beside this one directly, because a reader who knows them will ask what is left to do.

Table 1: The two closest benchmarks beside this one. The informative rows are the last three.

	GAMIBENCH	ORIGAMISPACE	CP2SEQ (OURS)
Model produces	A choice among rendered options	A predicted pattern, a relation, or crease-pattern code	A sequence of folds, one step at a time
Judged by	Agreement with the stored answer	Similarity to a stored target, or compilation	Execution of each step, then replay of the whole
Judge is	A stored key	A comparison against a reference artifact	The fold engine itself, run forwards
Judge can be wrong	Not applicable; no execution	Where similarity stands in for correctness	Only if the model of paper is wrong, which replay cannot detect (Sec. 4.8)
Wrong answers	186 authored impossible patterns	Not a designed axis	Unlimited: an illegal fold is produced by asking for one
Ground truth	Annotated per instance	Annotated per instance	Constructed; recorded as the fold happens
Non-unique answers	One key per question	Compared against one reference	Replayed and compared up to a declared symmetry group (Sec. 6.2)
Scores search effort	No	No	Yes: queries to solution, over all attempts (Sec. 6.3)

Read across the bottom three rows rather than the top. GamiBench’s impossible patterns are the closest the literature comes to forcing a judgement of feasibility rather than a recognition of shape, and they are authored, which is why there are 186 of them; here the same object is a by-product of generation and there is no limit on it. OrigamiSpace’s instrumentation per instance is richer than ours and is the reason 350 instances sustain a benchmark paper; what it does not have is a judge that can rule on a step the authors never anticipated. Neither ranks among the several valid folded states of one crease pattern, because both compare against a stored answer, and a stored answer cannot represent a set.

None of this is a defect in either benchmark. Both were built to measure whether a model understands folded geometry, and they measure it. The claim here is narrower: that measuring whether a model can *search* for a folding sequence requires a judge that executes, and that origami supplies one.

Collecting the metrics used across this literature makes one absence visible. Every metric in use compares a produced artifact against a target: geometric or semantic similarity to a reference, precision and recall against an expert sequence, IoU over affected edges, compilation success, human preference. **None ranks among the several valid folded states of a single crease pattern, and none scores an agent by how much verification it required.** The absence is structural rather than accidental, because computing any of those metrics requires a target to compare against, and the interesting question here has more than one right answer.

3 PROBLEM FORMULATION

3.1 FLAT-FOLDED STATES

A flat-folded state is an isometric map of the sheet into the plane together with a layer ordering. The map is fixed by the crease pattern; the ordering is not. A single crease pattern therefore admits many folded states that are geometrically identical and distinct as embeddings, and determining the ordering is NP hard even when a valid mountain-valley assignment is given (Bern & Hayes, 1996).

This distinction is the reason the task cannot be reduced to geometry. Over a folding sequence the crease graph’s topology never changes: the same vertices, edges and faces exist at step one and at step nineteen. What changes is everything else. Vertex coordinates are reflected at every step. The overlap structure, which faces lie above which, changes at every step and follows from the geometry. The layer ordering changes at every step and does *not* follow from the geometry; it is chosen, and it is where the decisions live. The number of layers accumulates, which is the formal content of “it gets harder as it goes”.

3.2 THE ACTION SPACE

Simple folds come in named variants, and any benchmark in this area has to say which it uses, because the complexity results differ between them.

Table 2: Named variants of the simple fold (Arkin et al., 2004).

MODEL	DEFINITION
One-layer simple fold	Rotate a single layer $\pm 180^\circ$ about a line
Some-layers simple fold	Rotate a contiguous run of layers
All-layers simple fold	Rotate every layer the line crosses, as sheet metal bends
Finite vs infinite line	Whether the fold line is a full line or a segment

CP2Seq instantiates the all-layers and some-layers tiers. The all-layers tier is the simpler object and it buys three properties at once, the third of which is rarely named. Because the whole stack moves as a rigid body, no two connected pieces of paper ever move relative to each other except at the fold line itself, where paper is allowed to bend. Self-intersection is impossible, the layer ordering is forced rather than chosen, and **tearing is impossible by construction.**

Moving only some layers brings all three back, and tearing becomes the binding constraint. If a moving face is joined to a stationary face along a crease, and that crease is not on the fold line, the fold would rip the sheet. This is decidable only in the coordinates of the original square, because adjacency is a fact about the original sheet that survives any amount of folding and cannot be recovered from the current positions of the polygons alone.

An example makes the tier difference concrete, and it needs only two folds. Fold the right half of a square over to the left across the vertical midline. The result is a two-layer stack whose top layer is joined to the bottom layer along the entire crease at the midline. Now select the top layer alone. Folding its upper half down across the horizontal midline would tear the sheet, because the moving piece is joined to stationary paper along a vertical crease segment that is perpendicular to the fold line. Folding its left quarter across a vertical line does not tear, because the moving piece's only join to stationary paper lies on the fold line itself. Same state, same layer selection, opposite verdicts, and the only difference is the orientation of the line.

——this 3 paragraphs are over emphasizing on whether all layers fold or some layer fold it gonna tear or not. is that the only point in this part????

3.3 THE TASK

Given a crease pattern, produce a sequence of simple folds that reproduces it. Generating an instance costs one forward pass of the engine; solving it is NP hard. That asymmetry is the paper's foundation.

It is tempting to say that legal moves are rare and that this is what makes the task hard. Enumeration refutes it: legal partial folds *grow* with depth, from 22 at two layers to 332 at thirty-eight, six times the 56 all-layers folds available at the same state. What falls is the hit rate of uniform random proposal, from 11.9% to 3.8% by seventeen layers, because the space being sampled grows faster than the legal set inside it. That is a fact about a sampler, not about origami, and conflating the two would put a false statement about branching factor into the paper.

——this paragraph should be listed properly and give a proper graph to illustrate it!!!!!!!

4 CP2SEQ: THE DATASET

4.1 WHY THE CORPUS IS SYNTHESISED

Every sample is built rather than collected, and the reason is that the construction supplies the label. A folding sequence is not something that can be read off a finished crease pattern by inspection, which is the whole premise of the task; so a corpus of annotated patterns would require either a solver, which is the thing being benchmarked, or a human expert, which does not scale and introduces an error rate nobody can measure. Folding forward and recording avoids both. The sequence is not inferred after the fact, it is what happened, and the pattern is what the folding left behind.

This also means difficulty is a dial rather than an observation. A collected corpus has whatever distribution of depth and entanglement its source happened to have. A generated one is stratified on purpose, and the empty regions of that stratification are visible rather than hidden (Section 4.7).

4.2 GENERATION, STEP BY STEP

The generator starts from a unit square and applies randomly chosen legal simple folds, recording each one, until it reaches a target depth. It then unfolds and reads off the crease pattern.

```
GENERATE(seed, depth_target, tier):
  rng   <- seeded RNG from 'seed'           # replay depends only on this
  state <- unit square, one layer
  seq   <- []

  while len(seq) < depth_target:
```

```

324     cands <- ENUMERATE_LEGAL_FOLDS(state, tier)    # all legal folds
325     if cands is empty:
326         return FAILED                            # dead end: discard, never repair
327     fold <- rng.choice(cands)
328     state <- APPLY(state, fold)                  # the APPLY the verifier uses
329     seq.append(fold: line, direction, layers, creases, coupling)
330
331     cp <- UNFOLD(state)                          # creases accumulated over the sequence
332     cp <- PLANARISE(cp)                         # split edges at crossings; canonical
333     frames <- [state after each step]           # retained as steps.fold
334
335     assert REPLAY(cp, seq) == state              # sample rejected if this fails
336     return Sample(cp, seq, frames, metrics(cp, seq))

```

Four properties of this procedure matter for the benchmark and each is a deliberate choice.

The generator never searches. It picks uniformly among legal folds and never backtracks. A dead end discards the sample rather than repairing it. This keeps generation one forward pass, and it is also why the recorded sequence carries no claim to being minimal (Section 6.1).

ENUMERATE_LEGAL_FOLDS and APPLY are the same functions the environment exposes to the model. The generator is not a privileged path through different code; it is the environment driven by a random policy. A sample is therefore reachable by definition, using exactly the action space the model is given.

Planarisation is canonical. A crease pattern is stored with every edge split at every crossing, so two patterns describing the same geometry have the same edge set and can be compared as multisets without a tolerance.

Every sample is replayed before it is kept. The assertion is not a test that runs sometimes; a sample that does not reproduce its own state is discarded at generation time.

4.3 REPRODUCIBILITY AS A SHIPPING FORMAT

Given its seed and the generator version, a sample regenerates byte-identically. The corpus is distributed as a generator plus a manifest, not as a data archive: the manifest lists each sample’s seed, tier, target depth and a hash of the resulting crease pattern, and rebuilding is verification. A reader who wants ten thousand samples instead of six hundred, or a stratification we did not choose, runs the generator rather than asking us for a larger download.

4.4 DIFFICULTY, STRATIFIED RATHER THAN SAMPLED

Difficulty is set along two axes rather than measured after the fact. **Fold depth** is the primary axis: the number of folds in the recorded sequence. It is the axis the task’s hardness is defined on, because step k is constrained by steps 1 through $k - 1$. **Coupling** is the second: the number of creases a single fold creates, which is to say how many layers the fold line cuts. It is free to compute at generation time, since one fold cuts every layer it crosses and each cut becomes one crease in the unfolded square. Coupling is the measurable proxy for non-local dependency: a high-coupling fold writes creases into many layers at once, so a later fold cannot be reasoned about locally. The two axes are not independent, and the dependence is itself a finding (Section 4.7).

4.5 WHAT THE CORPUS CONTAINS - THIS PART ONLY NEED TO LET READERS KNOW THAT , ALL LAYERS FOLD(EASY, MEDIUM, HARD LEVELS) + SOME LAYERS FOLD

other things like generated or verified, we don’t need to mention, we only need to put it in hugging face or appendix

Table 3: The files shipped with each CP2Seq sample. Only the first two, and only the final frame of the second, are shown to the model.

FILE	CONTENTS
<code>cp.fold</code>	The crease pattern, planarised. The input
<code>steps.fold</code>	The pattern plus the folded state after each step, as one multi-frame file. The final frame is the other input ; the intermediate frames are held back
<code>seq.json</code>	Every fold: line, direction, selected layers, creases made, coupling. The ground truth , never shown to the model
<code>meta.json</code>	Difficulty metrics, degeneracy flags, seed and generator version

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen_figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 2 placeholder: one dataset sample end to end. Left, the crease pattern; centre, the final folded state as X-ray and exploded views; right, the withheld ground-truth sequence, one diagram per fold.

Figure 1: [DRAFT IMAGE] One CP2Seq sample. The crease pattern and the final folded state are given to the model; the fold sequence on the right is the ground truth and is never shown.

4.6 WHAT A SAMPLE CARRIES

4.7 TWO LIMITS THAT BELONG IN THE PAPER RATHER THAN AN APPENDIX. ——MISSING

Caveat. The first is an empty cell. Long sequences at low coupling are almost unreachable, yielding 2hits in 16,000 attempts, because every all-layers fold thickens the stack, so a long sequence cannot keep cutting few layers. Real folders reach that corner by **pre-creasing**, an operation this action space excludes. The longer a real model runs, the more of it sits outside what the benchmark can express.

Caveat. The second is depth. The some-layers tier stops at six folds in this release, and the difficulty the benchmark is about lives past the depth a search can reach. Restoring the deeper tier is a matter of generator configuration and wall-clock, not redesign.

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen_figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 3 placeholder: the difficulty grid. Fold depth against coupling, shaded by sample count, with the long-sequence low-coupling cell visibly empty.

Figure 2: [DRAFT IMAGE] The difficulty grid. The empty corner is long sequences at low coupling, which this action space cannot reach because it excludes pre-creasing.

4.8 VERIFYING THE CORPUS ITSELF

Two checks run over every sample. Both replay the recorded sequence and compare the creases it makes against the pattern stored beside it; what they do not share is the comparison. The first groups edges into lines, merges collinear pieces and matches them within a tolerance. The second was written after five false failures had come out of exactly that machinery, each one a defect in the comparison rather than in the corpus, and reuses none of it. It merges each line’s pieces into maximal creased intervals, the quantity subdivision cannot change, and matches those one to one. Comparing subdivided segments instead does not work: of 457 failures under that earlier design, 364 were two sides carrying a different *number* of segments, which is not a distance and no tolerance can reach. The verdict is an absolute test at the radius at which the generator’s own planarizer identifies two points as one vertex, $1e-9$, the floor of what the corpus records rather than a constant tuned until the run passed. A depth-scaled ULP distance is reported alongside as a diagnostic on how much of that margin is used; on a sampled subset the median match needed 16 ULP, four orders inside the floor. Both checks pass on all 600 samples of the release corpus.

5 THE ENVIRONMENT, WHICH IS ALSO THE VERIFIER

5.1 ONE OBJECT, TWO NAMES

In this benchmark the environment and the verifier are the same object. Stepping it is the same act as asking for a verdict: the model submits a candidate fold, and the reply is either the resulting state or a refusal. There is no separate validity oracle, no second artifact that could disagree with the first, and no learned component in the loop.

5.2 THE DIVISION OF LABOUR

Table 4: Who does what. The environment does not search.

COMPONENT	RESPONSIBILITY
Environment / verifier	State update and legality. It does not search and does not choose
Model	Proposes the next fold, decides where to go, recognises when to backtrack

This division is what makes the benchmark measure something. The obvious objection to any tool-augmented setup is that the tool is doing the work, and here the answer is structural rather than rhetorical: deciding global flat-foldability is NP hard, so the environment cannot search its way to an answer even in principle. It can only say whether one proposed step is something paper could do. Compressing an exponential search into a feasible number of queries is exactly what the environment cannot supply and exactly what the model is being measured on. The referee is not a player.

5.3 REFUSALS ARE NAMED, ISN’T THAT YOU SHOULR MENTION A CHECK, EACH STEP CHECK AND FINAL STEP CHECK, FINAL STEP CHECK NOW IS MISSING , SHOULD MENTION HERE———

An illegal fold returns a structured refusal with a name, not a boolean.

The names are the design. A model told `would-tear` can act on the information; a model told “illegal” cannot. This also turns error analysis into a taxonomy that exists before the experiment: every rejection arrives already labelled with the class it violated, so a reject is a diagnosis rather than a dead end.

Final step check. - The four constraint classes that Flat-Folder checks (taco-taco, taco-tortilla, tortilla-tortilla, transitivity) (Akitaya et al., 2024; Flat-Folder) remain the right vocabulary for global terminal states; they are not what a single legal step needs checking against here.

Table 5: The refusal taxonomy. A rejection arrives already labelled with the class it violated.

REFUSAL	MEANING
would-tear	The moving layers are joined to stationary paper away from the fold line, so the sheet would come apart. The binding constraint of the some-layers tier
no-crease	The fold moves layers without creasing anything, tearing them free of the sheet rather than folding them
nothing-to-move	The fold line misses the selected layers entirely
direction-impossible	A run taken from the bottom of the stack was asked to fold over, or the reverse
bad-line	The fold line is malformed, or names no side of itself

5.4 WHAT THE MODEL OBSERVES - YOU SHOULD ADD A SCREEN SHOT OF SIMULATOR SOFTWARE !!!!!!!!!

On success the environment returns the new state together with two views: a top-down X-ray of the stack, and an exploded view of the layers. Every intermediate state in this tier is flat, so a second camera angle would show the same silhouette rotated and would carry no new information; what carries information is layer structure, which is why the second view is an explosion rather than a rotation. Positions are reported as coordinates on the original sheet together with an integer layer index. There is no continuous vertical coordinate at this tier, and a rendering that looks three-dimensional is not evidence of one; thickness belongs to a different problem.

6 SCORING

6.1 WHY THE OBVIOUS PROTOCOL IS WRONG

The recorded sequence is *a* solution, not *the* solution. The generator folds; it does not search, so nothing makes its sequence minimal, and no minimal-length label is shipped because producing one would require a search this benchmark deliberately does not run. Measured on the verified split, 2 of 100 samples admit a sequence one fold shorter than the one that built them.

The consequence is sharp. **Any metric that compares a proposal step by step against the recorded sequence marks a better answer wrong.** A model that finds a shorter correct solution is penalised for it. This disqualifies edit distance, and it disqualifies step-level precision, recall and F1 against an expert sequence for this task. Step count is not a correctness signal in either direction.

6.2 WHAT WE DO INSTEAD

Proposals are scored by replaying them through the engine and comparing the resulting states. Both sides are replayed: the model’s sequence and the reference sequence both pass through the same engine, and a stored frame is never the object compared against. That last point is a consequence of the file format rather than a preference. Stored frames record where each face sits in the current plane and not which piece of the original sheet it is, and deep in a stack many layers are congruent triangles, so two states differing by a swap of two such layers cannot be distinguished from stored frames alone. A replayed state carries original-sheet coordinates for free, because the engine needs them to decide tearing at all.

Two levels of equality are reported.

Level 1, crease sets. An exact multiset comparison of maximal creased intervals, equal rather than overlapping, with no tolerance in the verdict. This level is genuinely tolerance-free: edges are clustered into lines first, so subdivision differences cannot register as disagreements, and what remains is a combinatorial comparison.

Level 2, folded states up to a declared group of transforms. Because one crease pattern admits many valid terminal states, exact equality against the one stored state would mark correct answers

wrong. The answer is an equivalence: declare in advance which differences do not count, then demand equality inside that.

The transforms are quotiented out of every result reported in this paper. The declared group is the full plane isometry group: **arbitrary translation, arbitrary rotation, and reflection**. One single isometry must carry every layer of the candidate onto its reference counterpart; a match is not accepted by transforming each layer independently.

Each transform is in the group for a concrete reason rather than for generality. *Translation*, because a fold can carry paper off the original square and which half of the sheet travels decides where the stack lands; a folded model is the same model wherever it sits on the table. *Rotation*, because the same object produced by a sequence that worked around the square in a different order arrives rotated; rotations preserve both stack order and face parity, so they need no further correction. *Reflection, with two corrections*, because a reflection represents turning the model over, and turning a model over mirrors the coordinates, **reverses the stack** and **flips every face’s parity** at once. All three are applied together. Applying a reflection to coordinates alone would compare a model against its mirror image and silently accept wrong answers, which is the most dangerous failure a comparator of this kind can have.

Floating point is unavoidable once transforms are admitted, and this paper states its tolerance rather than claiming not to have one. A rotation by an angle that is not a multiple of a right angle produces irrational coordinates, so candidate and reference vertices agree only to within floating-point representation, and comparing them for bitwise equality would reject correct answers. The comparator matches polygons under a tolerance of 2×10^{-6} in the plane, with degenerate-vertex cleanup at 10^{-7} and a collinearity threshold of 10^{-9} . These values are reported in every result record, so a reader can see what was used rather than infer it.

Caveat. This qualifies a claim made elsewhere. Level 1 crease-set comparison is genuinely tolerance-free, since the verdict involves no floating-point threshold. Level 2 is not, and cannot be, because it admits rotations. The precise formulation is that the *verifier* is exact, since legality is decided combinatorially by the engine, while *terminal-state equality under transforms* carries a stated numerical tolerance.

Caveat. The layer-order variants that a flat-foldability solver reports as equally valid are deliberately *not* in the group. Those are not symmetries of one state; they are different states reachable from the same crease pattern, and deciding which orderings are valid requires a solver. Folding that into an equality test would hide an external dependency inside what reads as arithmetic. If that equivalence is wanted, it belongs in a separate check that owns the solver.

6.3 METRICS

The primary metric is **solve rate at a fixed query budget**, reported per difficulty stratum and never pooled, under both levels of equality. A proposal is solved if replaying it reproduces the target. Exhausting the budget is a timeout, which is reported separately from a wrong answer because the two mean different things.

The secondary metric is **queries to solution**, reported over all attempts including those that time out. Conditioning on the solved attempts hides precisely the long-tail blowup that makes the task interesting. This metric is what answers the “the simulator did all the work” objection quantitatively: two models sharing one exact verifier can be ranked by how little of it they needed.

6.4 VALIDATING THE PROTOCOL

A verifier paper that does not test its own verifier is asking to be taken on trust, so the comparator ships with a negative control: a deliberately corrupted state, with two layers swapped, must be rejected.

The control earned its place. On its first full run, 23 of 600 corrupted states were **accepted**. The cause was not a tolerance. The comparison used current-plane geometry and parity alone, and deep in a stack many layers are congruent triangles, so two layers occupying the same region but originating from different parts of the sheet were interchangeable. They are not interchangeable: which one lies underneath is a real difference between two folded states, and precisely the difference

Level 2 exists to catch. With face identity on the original sheet restored to the comparison, the full corpus reports 600 states equal, all under the identity symmetry, and 600 of 600 corrupted states rejected. Every verdict additionally reports whether face identity was available, so a weakened comparison is visible in the output rather than assumed away.

7 EXPERIMENTAL SETUP

7.1 THE PIPELINE —SHOULD HAVE A HAND DRAWN GRAPH

One episode is one sample attempted by one model under one tier of assistance. The loop is the same for every arm and every model, and the only thing that varies between arms is what the model is allowed to see and call.

```
EPISODE(sample, model, tier, budget):
  cp, final <- LOAD(sample)      # cp.fold, and the LAST frame of
                                # steps.fold; other frames withheld
  state <- unit square, one layer
  seq <- []
  while len(seq) < budget:
    obs <- RENDER(state, tier)    # withheld at the no-vision tier
    action <- model(cp, final, obs, history, tier)
    if action is SUBMIT: break
    verdict <- ENVIRONMENT.step(state, action) # verifier IS this
    if verdict is REFUSAL:
      history.append(refusal with its reason) # state unchanged
    else:
      state <- verdict.state
      seq.append(action)
      history.append(obs)
  return SCORE(replay(cp, seq), replay(cp, reference_seq))
```

Three things about this loop decide what the benchmark measures.

The environment is the only thing that changes state. A model cannot assert that a fold happened. It proposes, and the environment either performs the fold or refuses it, so the state the model reasons about is always a state paper could be in.

A refusal costs a query and does not advance the episode. This is what makes queries-to-solution meaningful: a model that proposes carelessly pays for it in budget, and a model that reasons before proposing is rewarded in the secondary metric even when both eventually solve the sample.

The reference sequence is never read during the episode. It enters only at scoring time, and then only as something to replay, never as something to compare against step by step (Section 6.1).

7.2 THE TOOLS THE MODEL IS GIVEN

The tool belt is the experimental variable. Every tool is a read or a step against the same engine; none of them searches, and none of them ranks candidates.

`list_legal_folds` is the tool that needs justifying, because it is doing the most work. It enumerates candidate actions and returns those that are legal, which is a large prune performed by the simulator rather than by the model. That is precisely why the tier structure exists and why the deterministic baseline of Section 7.4 is reported: if exhaustive search over the enumerated set solves the corpus outright, then the filtered arm is not measuring origami reasoning, and the benchmark has to say so rather than let a headline number imply otherwise.

7.3 TIERS, MODELS AND PROTOCOL

Tiers. Four levels of assistance, from a full tool belt down to none; the table is in Section 8. The same model, the same samples and the same query budget across tiers, so that the only difference between two tiers is what was withheld.

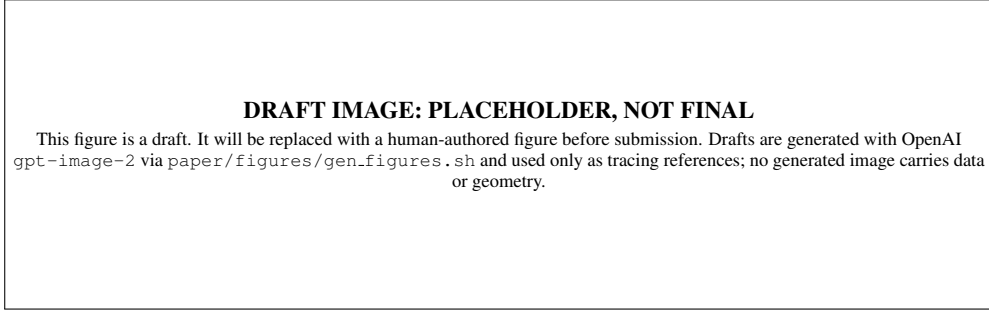


Figure 4 placeholder: the evaluation pipeline. Inputs on the left; the propose-verify-observe loop in the centre with the environment and verifier drawn as one box and refusals looping back without advancing; replay and scoring on the right, with the reference sequence outside the loop.

Figure 3: [DRAFT IMAGE] One episode. The environment and the verifier are the same object, so stepping is the same act as asking for a verdict. The reference sequence enters only at scoring.

Table 6: The tool belt. Tiers are defined by which of these are available.

TOOL	WHAT IT RETURNS	WHY IT EXISTS
<code>apply_fold</code>	The new state, or a named refusal	The action. This call is the verification
<code>get_views</code>	Top-down X-ray of the stack, exploded layer view	The visual channel; withheld at the no-vision tier
<code>get_state</code>	Face positions in original-sheet coordinates plus integer layer index	The symbolic channel, so a model is never forced to read geometry off a raster
<code>list_legal_folds</code>	Every fold the engine would currently accept	The filtering tier. It prunes; it does not choose
<code>undo</code>	The previous state	Backtracking without spending the episode

Models. [TO FILL from the aggregation: the models actually run, their versions, and the reasoning-effort and image-history settings used. Held fixed across tiers, because comparing across models and across tiers at once would confound the two.]

Protocol. Query budget, number of repeats and seeds are fixed before the first run and reported with the results. Median and spread across repeats are reported, never a single run, because sampling is not deterministic and one run is not a measurement.

Transforms are applied in every reported result. Every solve rate in Section 9 is computed after quotienting out the plane isometry group of Section 6.2: translation, rotation and reflection, with reflections reversing the stack and flipping face parity. A candidate is counted correct if one single isometry carries every layer onto its reference counterpart. This is not a post-hoc leniency applied to borderline cases; it is part of the definition of a correct answer, applied identically to every arm, every model and every baseline. Turning it off would mark correct answers wrong, because where a folded model lands on the table is an accident of which half of the sheet travelled.

Matching under transforms requires a floating-point tolerance, and it is stated. Admitting arbitrary rotation means candidate and reference coordinates agree only to within floating-point representation, so the comparator matches polygons within 2×10^{-6} , cleans degenerate vertices at 10^{-7} and treats vertices as collinear below 10^{-9} . Every result record carries the tolerance it was scored under, so the value is visible in the released runs rather than buried in a constant. The engine’s

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen_figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 5 placeholder: a refusal as the model receives it. The proposed fold line over the current state, the named reason would-tear, and the offending crease segment highlighted; beside it the same state with a legal fold line.

Figure 4: **[DRAFT IMAGE]** A refusal is a diagnosis, not a dead end. The same state and the same layer selection give opposite verdicts depending only on the orientation of the fold line.

legality decisions are combinatorial and involve no tolerance; the numerical threshold applies only to terminal-state equality under transforms.

Error verbosity is itself a variable. The richer a refusal, the more of the reasoning the environment performs rather than the model; at the limit a message that names the fix has solved the step. Verbosity is fixed before the first run and reported, otherwise the comparison between tiers measures message design rather than reasoning.

7.4 BASELINES

Three, none of them a language model, so the models are ranked against something rather than only against each other.

Table 7: The baseline ladder. [TO RUN]

BASELINE	WHAT IT ESTABLISHES
Uniform random over enumerated legal actions	The floor. What the tool alone achieves with no reasoning at all
Deterministic breadth-first over the same action set (workspace/search_baseline.mjs)	How much of the task the enumerator already solves, and the depth at which exhaustive search stops being affordable
Symbolic crease-pattern-to-sequence solver (Creasy)	The closest thing to prior art on the task itself

The breadth-first baseline is the one that answers the “the simulator did all the work” objection of Section 7 numerically rather than structurally, which is the stronger form of that answer.

8 PROGRESSIVE TIERS OF TOOL ASSISTANCE

[TO RUN]

The model is run at four levels of assistance, each removing one kind of help from the level above. These are reported as tiers rather than as ablations, for the reason given in Section 16: a full ablation study was planned and has not been run, and calling a partial sweep an ablation would overclaim.

Same model, same patterns, same query budget across tiers. If the no-vision or no-tools tier performs nearly as well as the full tier, **that is a finding rather than a failed experiment**: it means the gain is not where it was assumed to be. [TO RUN]

Table 8: The tiers of assistance. Same model, same patterns, same query budget across tiers.

TIER	TOOLS AVAILABLE	WHAT IT ISOLATES
Full tool belt	Environment, rendered views, any auxiliary tools	Upper bound
No vision	Same, rendered views withheld	The contribution of visual feedback
Verifier only	Pass/fail, no filtering	The contribution of filtering on top of raw verification
No tools	None, anonymized geometry	Memorization control

9 RESULTS

All numbers below come from `workspace/RESULTS/results.md`, produced by `python3 workspace/aggregate_results.py` over every saved run. 1,211 attempts across 47 runs and 256 distinct samples. Every attempt is counted once, and an attempt with no recorded `solved` field is counted as unsolved.

9.1 HEADLINE

No language model solved a single sample beyond the easy stratum. Across every model arm, mid is 0 of 70 and hard is 0 of 27. On the easy stratum a deterministic breadth-first search over exactly the action set the models are given solves 54.7%, against 20.4% for the best-covered model at the same tier. **Blind search beats every model tested, by a factor of about 2.7 on the only stratum where anything succeeds at all.**

Table 9: Solve rate per stratum, all saved runs. Denominators are attempts, not distinct samples. Rows with fewer than ten attempts support no rate and are shown only so that reporting is not selective.

ARM	TIER	EASY	MID	HARD	OVERALL	CALLS
deterministic-bfs	legal-folds	366/669 (54.7%)	1/40 (2.5%)	0/16	367/725 (50.6%)	5
gpt-5.6-luna	legal-folds	59/289 (20.4%)	0/66	0/25	59/380 (15.5%)	23.5
gpt-5.6-luna	no tools	4/65 (6.2%)	0/1	0/1	4/67 (6.0%)	29
claude-sonnet-5	legal-folds	0/5	–	–	0/5	10
gpt-5.6-sol	no tools	0/7	–	–	0/7	20
gpt-5.6-terra	no tools	0/1	–	–	0/1	35
gpt-6-astra	no tools	1/1	–	–	1/1	5
codex-cli-chatgpt	none	0/1	–	–	0/1	2
<i>errored, model unrecorded</i>	–	0/20	0/3	0/1	0/24	–

Caveat: read the denominators before the percentages. Only `gpt-5.6-luna` and the baseline have coverage worth a rate. `gpt-6-astra` shows 1/1; **that is one attempt and must not be reported as 100%**. `claude-sonnet-5`, `gpt-5.6-sol`, `gpt-5.6-terra` and `codex-cli-chatgpt` have between one and seven attempts each. These rows are present because omitting them would be selective reporting, not because they support a comparison. Any main table in the submitted paper prints *n* in every cell or drops the row.

9.2 THE BASELINE READING

`deterministic-bfs` solves 367 of 725 attempts overall. Per stratum: 366/669 easy, 1/40 mid, 0/16 hard. Median states expanded is 65 on easy, 1051.5 on mid, 532 on hard. 358 of its 725 attempts ended in timeout rather than exhaustion, so **50.6% is a lower bound under the baseline’s time budget, not the ceiling of exhaustive search.**

This is the outcome Section 16.7 anticipated and the one `workspace/search_baseline.mjs` was written to detect. Its header records that `list_legal_folds` evaluates roughly 2,010 candidate actions per state and returns a mean of 3.51 that are legal and stay inside the target crease pattern, over 73,750 enumerations. A branching factor near 3.5 makes an easy sample of five folds a tree of a few hundred states, and the median of 65 states expanded on easy confirms it. **The easy stratum is therefore not measuring origami reasoning.** It measures whether a model can avoid losing to breadth-first search on a tree its own tool has already pruned roughly 570-fold, and every model tested loses.

We state this rather than report the models’ easy-tier rate as an achievement. It is also the answer to the “the simulator does all the work” objection of Section 7, and the answer is uncomfortable: on the easy stratum the simulator does do most of the work. The benchmark’s value here is that it measures that rather than hiding it. On mid and hard the objection dissolves, because search solves 1 of 56 and the models solve 0 of 97.

9.3 WHAT THE TOOLS CONTRIBUTE

The one within-model tier comparison the data supports is `gpt-5.6-luna` on easy: 59/289 (20.4%) with the filtering tool against 4/65 (6.2%) with no tools, so filtering is worth roughly 3.3× on that stratum and nothing on mid or hard, where both tiers are zero.

Caveat. The no-vision and verifier-only tiers of Section 8 were not run. The four-tier ladder this paper describes is not yet instantiated; only its two endpoints exist.

9.4 FAILURE TAXONOMY

Table 10: Termination reasons for `gpt-5.6-luna`, 447 attempts.

TERMINATION	<i>n</i>	SHARE
<code>state_cycling</code>	215	48.1%
<code>finished</code>	167	37.4%
<code>turn_budget</code>	47	10.5%
<code>repetition_detected</code>	18	4.0%

This is the most informative result in the section, and it is not a solve rate. A majority of attempts, 233 of 447 counting cycling and repetition together, end with the model revisiting a state it has already produced rather than exhausting its budget. The model is not running out of room to search; it is failing to notice it has been somewhere before. That is a specific, diagnosable deficit in maintaining state across a multi-step spatial task, and it is the kind of finding a benchmark with an executing verifier can produce and a multiple-choice benchmark cannot. The baseline never cycles, because breadth-first search over a visited set cannot; it times out instead, 358 times.

9.5 WHERE DIFFICULTY LIVES

Depth separates the arms completely. Easy is the only stratum where anything succeeds; mid yields a single solve, by the baseline, at a median of 1051.5 states expanded against 65 on easy; hard yields none from any arm. [TO RUN: solve rate against coupling, to test whether the second stratification axis of Section 4.4 predicts difficulty as intended. Only depth is evidenced so far.]

9.6 THREATS TO THESE RESULTS

Every model attempt ran at low reasoning effort. All 461 language-model attempts with a recorded setting used `reasoning_effort: low`; 25 have none recorded. This is the single largest caveat on every claim in this section. “Frontier models fail this task” is not supported by these runs. What is supported is that frontier models at low reasoning effort fail this task and lose to

breadth-first search where search works at all. Re-running the best-covered arm at high effort is the most valuable remaining experiment, and until it exists the headline carries the qualifier.

Coverage is severely imbalanced. 725 baseline attempts and 447 for gpt-5.6-luna, against between one and seven for every other model. No cross-model claim is made.

24 attempts terminated in error with no model recorded and are counted as unsolved. They are reported as their own row rather than dropped, because dropping failed attempts inflates every rate above them.

Repeats are not yet reported as median and spread. Section 7.3 promises this; the aggregation counts attempts rather than grouping repeats per sample. The protocol requires it before submission.

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen_figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 6 placeholder: solve rate per difficulty stratum, one line per model, with the deterministic baseline as a distinct reference line crossing above every model line on the easy stratum, and attempt counts printed at each point.

Figure 5: [DRAFT IMAGE] Main result per stratum. The baseline crosses above every model line on easy; that crossing is the most informative feature of the chart and is not smoothed away.

10 LIMITATIONS

Every model attempt ran at low reasoning effort. All 461 language-model attempts with a recorded setting used `reasoning_effort: low`. The results support a claim about models at that setting, not about frontier models in general, and the headline is worded accordingly. Raising the effort on the best-covered arm is the most valuable remaining experiment.

The easy stratum does not measure what the benchmark is for. Deterministic breadth-first search solves 54.7% of it at a median of 65 states expanded, beating every model. The stratum measures whether a model can beat trivial search on a tree its own tool has pruned roughly 570-fold. Claims about reasoning should be read off mid and hard, where every arm including search is at or near zero, and where the benchmark currently discriminates nothing either. **The band in which this benchmark separates models may be narrow or, at this depth range, empty.**

Model coverage is severely imbalanced. 447 attempts for one model and between one and seven for four others. No cross-model comparison is supported.

No training. Every arm is an off-the-shelf model driven by prompting and tool calling; no model is trained or fine-tuned. The contribution is the harness, not a model. This is stated in the introduction as well as here, because a limitation the authors declare is context and one a reviewer discovers is an objection.

Scope of the action space. 89.3% of the 366 real crease patterns in Flat-Folder’s `examples/instagram/` corpus lie outside all-layers simple folding, and pre-creasing, the operation real folders use to reach long sequences at low coupling, is excluded.

The corpus verifiers share an engine with the generator, so they cannot detect a wrong model of paper. They detect drift between what was folded and what was recorded.

Flat states only. No thickness, no material, no mechanics. Thickness changes the geometry outright and belongs to a separate line of work with a verifier several orders of magnitude more expensive.

Level 2’s group is narrower than the truest notion of equality, because valid layer-order variants are excluded for the reason given in Section 6.2. It is therefore strict rather than permissive, which is the safe direction but not a free one.

11 FIGURES

Table 11: Planned figures and their status.

#	FIGURE	STATUS
1	The asymmetry: generation is one forward pass, recovery is NP hard	To draw
2	The tearing pair of Section 3.2. One state, one layer selection, a vertical fold line legal and a horizontal one not	To draw
3	A refusal as the model receives it: view, named reason, per-constraint checklist with locations	To draw
4	The difficulty grid, depth against coupling, with the empty cell visible	To draw
5	Main result per stratum	[TO RUN]

12 REPRODUCIBILITY

The corpus ships as a generator and a manifest; every sample rebuilds byte-identically from its seed. The environment, both corpus verifiers and the state comparator are released with the benchmark. The negative control of Section 6.4 runs as a command, so a reader can confirm that the comparator rejects corrupted stacks rather than taking Section 6.4 on trust.

13 LICENCE AND AVAILABILITY

The corpus generator, the environment and verifier, both corpus checks, the state comparator and the evaluation harness are released under the **BSD 3-Clause** licence.

Code and data are released on publication; links are omitted here.

The artifacts carry no personal data, no human-subject data and no scraped content. Every sample is synthesised from a seed by folding a square, so no licensing question attaches to the geometry and no attribution is owed to any origami designer. Flat-Folder’s `examples/instagram/` set, the one external corpus this paper measures against, is not redistributed; the scope measurement of Section 10 was computed against it in place and is reported as a number.

14 USE OF LARGE LANGUAGE MODELS

ICLR 2026 requires that **any** use of a large language model be disclosed, on the Code of Ethics principle that all contributions to the research must be acknowledged, and holds the authors responsible for everything in the paper regardless of what assisted in producing it. The policy does not treat a model used as a research instrument differently from one used for writing: both must be disclosed. Disclosure is required **in the paper’s text and in the submission form**, so filling in this section is not sufficient on its own.

Caveat. Remember the submission form. It is a separate field and is easy to miss.

Large language models are part of the object of study. This is a benchmark for evaluating language models, so models appear in the work by construction. Every number reported in Section 9 is the output of a frontier multimodal model driven through the harness described in Section 7. The models evaluated, their versions and their settings are listed there. No model was trained or fine-tuned for this paper.

Code. The benchmark’s implementation was written with AI assistance, principally Claude Code, used as a coding assistant throughout. Every design decision, every experiment, and the direction of the work were determined by the human authors, and all generated code was reviewed by them before it was used. The authors are responsible for the correctness of the released artifacts. Two habits in the project exist because of this reliance and are worth naming: the corpus carries two independently written checks over every sample (Section 4.8), and the state comparator ships with a negative control that must reject a deliberately corrupted state (Section 6.4). The second of these caught a real defect, described in that section, in which 23 of 600 corrupted states were initially accepted.

Literature search. The related work of Section 2 was assembled primarily by hand, using Google Scholar and alphaXiv. AI assistance was used partially, to locate candidate papers and to recover bibliographic details. Every citation in this paper was verified against the arXiv or publisher record by a human author; citations that could not be verified were removed rather than softened.

Writing. AI assistance was used in drafting and editing the manuscript. The argument, the claims, the experimental design and the conclusions are the authors’. No text was included that an author had not read and agreed with, and no result, citation or number in this paper was produced by a language model without being checked against the artifact it describes.

Figures. Any figure marked **DRAFT IMAGE** is a placeholder generated with OpenAI gpt-image-2 and is used only as a compositional reference to be redrawn by hand. Generated images are never used to depict data or geometry: crease patterns, folded states and result charts are rendered from the corpus and from the run outputs. Draft figures will not appear in a submitted version of this paper.

ACKNOWLEDGMENTS

Withheld for anonymous review.

REFERENCES

- Naaisha Agarwal, Yihan Wu, Yichang Jian, Yikuan Hu, Nishad Mansoor, Mohan Li, Yifei Peng, Wang-Zhou Dai, Yao-Xiang Ding, and Emanuele Sansone. OrigamiBench: An interactive environment to synthesize flat-foldable origamis. *arXiv preprint arXiv:2603.13856*, 2026. VERIFIED 2026-09-21 against the arXiv record.
- Hugo A. Akitaya, Jun Mitani, Yoshihiro Kanamori, and Yukio Fukui. Generating folding sequences from crease patterns of flat-foldable origami. In *ACM SIGGRAPH Posters / Student Research Competition*, 2013.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Simple folding is really hard. *Journal of Information Processing*, 25:580–589, 2017.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Computing flat-folded states. In *Origami⁸: Eighth International Meeting on Origami in Science, Mathematics and Education (OSME)*, 2024.
- Hugo A. Akitaya et al. Infinite all-layers simple foldability. *Graphs and Combinatorics*, 2019. arXiv:1901.08564.
- Esther M. Arkin, Michael A. Bender, Erik D. Demaine, Martin L. Demaine, Joseph S. B. Mitchell, Saurabh Sethia, and Steven S. Skiena. When can you fold a map? *Computational Geometry*, 29(1):23–46, 2004.
- Marshall Bern and Barry Hayes. The complexity of flat origami. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 175–183, 1996.
- Creasy. Creasy, 2022. Open-source crease-pattern-to-sequence implementation, Java, GPL-3.0, unmaintained since 2022. <https://github.com/xkevio/Creasy>.

- Erik D. Demaine and Joseph S. B. Mitchell. Reaching folded states of a rectangular piece of paper. In *Proceedings of the 13th Canadian Conference on Computational Geometry (CCCG)*, 2001.
- Erik D. Demaine, Satyan L. Devadoss, Joseph S. B. Mitchell, and Joseph O’Rourke. Continuous foldability of polygonal paper. In *Proceedings of the 16th Canadian Conference on Computational Geometry (CCCG)*, 2004.
- Flat-Folder. Flat-Folder, 2024. <https://github.com/origamimagiro/flat-folder>, MIT licence. Decides and enumerates flat-folded states.
- Flat Origami is Turing Complete. Flat origami is Turing complete. *arXiv preprint arXiv:2309.07932*, 2023.
- FOLD file format. The FOLD file format, 2019. <https://github.com/edemaine/fold>.
- Foldings and Meanders. Foldings and meanders. *arXiv preprint arXiv:1302.2025*, 2013.
- Graphs of Origami Flat Foldings. Realization and connectivity of the graphs of origami flat foldings. *arXiv preprint arXiv:1808.06013*, 2018.
- Y. Huang, Y. Chen, Y. Jiang, J. Han, Z. Tu, Y. Yang, and C. Jiang. Learn2Fold: Structured origami generation with world model planning. *arXiv preprint arXiv:2603.29585*, 2026.
- John E. Koehler. Folding a strip of stamps. *Journal of Combinatorial Theory*, 5:135–152, 1968.
- Layer Ordering Constraints. An algebraic approach to layer ordering constraints for origami flat-foldability. In *Origami*⁸, 2026.
- Édouard Lucas. Théorie des nombres, 1891. Attributes the stamp-folding problem to É. Lemoine; earlier literature traced by J. Touchard (1950).
- W. F. Lunnon. Multi-dimensional map folding. *The Computer Journal*, 1971.
- Jun Mitani. ORIPA: Origami pattern editor. Crease-pattern editor and folded-form estimation.
- M. Moriya et al. FoldingAgent. *arXiv preprint arXiv:2609.00377*, 2026. Reconstructs folding processes from instructional video; built on PurelandFold.
- OEIS Foundation Inc. The on-line encyclopedia of integer sequences, A000136: Number of ways of folding a strip of n labeled stamps, 2026. See also A001011, A001416.
- PurelandFold. PurelandFold, 2026. <https://huggingface.co/datasets/mayaweiz/PurelandFold>, CC-BY-4.0. 27 sequences, 337 frames.
- Random Locally Flat-Foldable Origami. On random locally flat-foldable origami. *arXiv preprint arXiv:2502.04279*, 2025.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- Simple Folding of Mixed Orthogonal CPs. Complexity of simple folding of mixed orthogonal crease patterns. *arXiv preprint arXiv:2306.00702*, 2023.
- Spatial Reasoning in MLLMs. Spatial reasoning in MLLMs: A survey. *arXiv preprint arXiv:2511.15722*, 2025.
- R. Spencer, R. Yaari, R. Vemavarapu, J. Yang, S. Ngo, and U. Sharma. GamiBench: Evaluating spatial reasoning and 2D-to-3D planning capabilities of MLLMs with origami folding tasks. *arXiv preprint arXiv:2512.22207*, 2025. VERIFIED 2026-09-21: 186 regular and 186 impossible patterns, six viewpoints, three VQA tasks, VC and IFSR metrics.

Spin Model for Flat-Foldability. A spin model for global flat-foldability of random origami. *arXiv preprint arXiv:2403.07306*, 2024.

Visualization-of-Thought. Mind’s eye of LLMs: Visualization-of-thought elicits spatial reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. *arXiv:2404.03622*.

R. Xu, D. Lu, Z. Zhao, X. Tan, X. Wang, S. Yuan, J. Chen, and Y. Xu. OrigamiSpace: Benchmarking multimodal LLMs in multi-step spatial reasoning with mathematical constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. VERIFIED 2026-09-21: NeurIPS 2025 poster; also *arXiv:2511.18450*.

Tom Zahavy, Shaobo Hou, Thomas Tumiel, James Doran, Francesco Faccio, Xidong Feng, Alex Havrilla, Igor Khytryi, Chenglei Li, Lisa Schut, Vivek Veeriah, Arijan Abrashi, Michał Kosmulski, Robert J. Lang, Nick Robinson, Brandon Wong, Marcus Chiam, Gloria Fang, and Satinder Singh. COrigami: An AI pipeline for co-designing flat-foldable visually recognisable origami. *arXiv preprint arXiv:2606.26299*, 2026. VERIFIED 2026-09-21 against the arXiv record.

A REFERENCE NOTES

The bibliography is grouped in `iclr2026_conference.bib` by the role each work plays in the argument rather than alphabetically, so that a co-author can see what is load-bearing. The notes below record what is still outstanding.

Origami benchmarks and systems for language models. Xu et al. (2025) must be cited as a preprint; `papers.md` recorded NeurIPS’25 and the arXiv record carries no venue. Spencer et al. (2025) has code and data released. Moriya et al. (2026) reconstructs folding processes from instructional video and is built on the PurelandFold dataset; code at <https://github.com/maya-moriya/FoldingAgent>. **OrigamiBench** (Agarwal et al., 2026) has a *citation missing*: it appears in our metrics survey as the source of query efficiency and geometric/semantic similarity, and its dataset is the 366patterns in Flat-Folder’s `examples/instagram/`. A full reference has to be recovered before submission. **COrigami** (Zahavy et al., 2026) has a *citation missing* as well: it appears in the metrics survey as the source of flat-foldability as a boolean and a VLM aesthetic score. Same action needed.

Complexity and theory of flat folding. Bern & Hayes (1996) is the citation under the paper’s central claim: geometry does not determine layer order, and deciding overlap order is NP hard even given a valid mountain-valley assignment. Arkin et al. (2004) defines simple foldability; the action space comes from here. Akitaya et al. (2019) covers the all-layers model, which corresponds to sheet-metal bending. Akitaya et al. (2024) is the Flat-Folder paper; deciding a global flat-folded state is NP hard. See also Simple Folding of Mixed Orthogonal CPs, Layer Ordering Constraints and Graphs of Origami Flat Foldings. Flat Origami is Turing Complete is a defensive citation only, per Section 7. Demaine et al. (2004) separates folded state from folding motion: reachability is free, so the hard question is the discrete step structure; its predecessor is Demaine & Mitchell (2001).

The crease-pattern-to-sequence problem itself. Akitaya et al. (2013) named the problem, by the group whose tools the field now uses: reflection paths, graph rewriting, step graphs; a frog base explodes to 22,665 nodes and 30 minutes, and the authors’ own future work asks for the priority heuristic this paper’s models are being asked to supply. Creasy is an open-source implementation of the above and is usable as a symbolic baseline. [TO RUN if used]

Counting folded states. OEIS Foundation Inc. (2026) lists the foldings of a strip of stamps: 1, 2, 6, 16, 50, 144, 462, 1392, 4536, 14060, ... There is no closed form. See also A001011 and A001416, and Lucas (1891); Koehler (1968); Lunnon (1971); Foldings and Meanders; meanders and stamp foldings are the same combinatorial object.

Spatial reasoning context. Spatial Reasoning in MLLMs surveys the area. Visualization-of-Thought names the mental-imagery question this benchmark can turn into a measurement.

Tools, formats and data. Flat-Folder decides and enumerates flat-folded states and is the source of the four constraint classes cited in Section 5.3. FOLD file format is the de facto standard used by every artifact we ship. Mitani is a crease-pattern editor with folded-form estimation. PurelandFold is not used as training or evaluation data here; it is cited as the closest existing sequence-level corpus and as FoldingAgent’s data.

Statistical physics of random flat-foldability. Optional support for the claim that multiple valid states is the normal case rather than a curiosity, to be included only if Section 3.1 needs reinforcement: Spin Model for Flat-Foldability and Random Locally Flat-Foldable Origami.

Deliberately not cited. Thick folding (Ku & Demaine 2016), bar-and-hinge mechanics, MERLIN2, SWOMPS, Sim-FAST-PY, and the creased-sheet mechanics literature all belong to the thickness and mechanics line of work. They are a separate paper with a verifier several orders of magnitude more expensive, and citing them here would invite a reviewer to ask why this paper does not do that one.